

File Input and Output (File I/O) in Python

What is a File?

A **file** is a place on a computer where we can store data so that it remains available even after the Python program has stopped running. For example, a text file such as `notes.txt` can store notes, a CSV file such as `marks.csv` can store student marks, and an image file such as `photo.jpg` can store a picture.

In Python, **File Input and Output**, commonly called **File I/O**, is used to communicate with files. **Input** means taking or reading data from a file into the Python program. **Output** means sending or writing data from the Python program into a file.

For example, when we read a student's name from a file, that is **input**. When we save a student's name into a file, that is **output**.

Why Do We Use Files?

When we store information only in a Python variable, the information normally exists only while the program is running.

For example:

```
name = input("Enter your name: ")  
print(name)
```

The name entered by the user is stored in the variable `name`. When the program ends, the variable and its value are no longer available.

If we want to save the information so that we can use it later, we can store it in a file.

For example, if the user enters:

Aniketh

we can save `Aniketh` in `student.txt`. Even after the program finishes, the information remains inside the file.

Therefore, files are useful for **permanent data storage**.

Opening a File

Before Python can read from or write to a file, we need to **open the file**.

The basic syntax is:

```
file = open("filename", "mode")
```

For example:

```
file = open("sample.txt", "r")
```

Here, `"sample.txt"` is the name of the file we want to work with. The `"r"` tells Python that we want to **read** the file.

The word `file` is a variable. It does not contain the actual text immediately. Instead, it represents the **opened file**, through which Python can perform operations such as reading or writing.

Think of opening a file like **opening a notebook before reading or writing in it**. Once the notebook is open, we can perform the required operation.

File Modes

When opening a file, we specify a **mode**. The mode tells Python what we want to do with the file.

The commonly used modes are:

Mode	Meaning
<code>"r"</code>	Read
<code>"w"</code>	Write
<code>"a"</code>	Append
<code>"x"</code>	Create a new file
<code>"r+"</code>	Read and write

The mode is important because Python behaves differently depending on which mode we use.

Read Mode `"r"`

The `"r"` mode means **read**. It is used when we want to take existing data from a file and bring it into our Python program.

```
file = open("sample.txt", "r")
```

Here, Python opens `sample.txt` for reading. We can then use functions such as `read()` or `readline()` to get the content.

If the file does not exist, Python will normally produce a `FileNotFoundError`, because there is no existing file to read.

Write Mode "w"

The "w" mode means **write**. It is used when we want to put new data into a file.

```
file = open("sample.txt", "w")
```

There is one very important thing to remember about "w" mode. If the file already contains data, opening it in "w" mode **removes the existing content** before writing the new content.

For example, suppose the file contains:

Hello

Welcome to Python

If we run:

```
file = open("sample.txt", "w")
```

```
file.write("I am learning Python")
```

```
file.close()
```

the old content is replaced.

The file will now contain:

I am learning Python

So remember:

"w" = Write new content and overwrite old content.

If the file does not already exist, "w" can create the file.

Append Mode "a"

The "a" mode means **append**. It is used when we want to add new data to the existing content without deleting what is already there.

Suppose the file contains:

I am learning Python

Now we write:

```
file = open("sample.txt", "a")
```

```
file.write("\nPython is easy")
```

```
file.close()
```

The `\n` moves the new text to the next line.

The file will now contain:

I am learning Python

Python is easy

The old content remains, and the new content is added after it.

So remember:

"a" = Add new content without removing old content.

Create Mode **"x"**

The **"x"** mode is used to **create a new file**.

```
file = open("newfile.txt", "x")
```

This asks Python to create **newfile.txt**.

If a file with the same name already exists, Python produces an error instead of overwriting it.

This mode is useful when we specifically want to make sure that we are creating a new file.

Read and Write Mode **"r+"**

The **"r+"** mode allows us to **read and write** to an existing file.

```
file = open("sample.txt", "r+")
```

This mode is useful when the program needs to both read existing content and perform writing operations.

Reading Data from a File

Suppose our **sample.txt** file contains:

Hello

Welcome to Python

We can use Python to read this information.

Reading the Entire File Using **read()**

The **read()** function reads the contents of the file.

```
file = open("sample.txt", "r")
```

```
content = file.read()
```

```
print(content)
```

```
file.close()
```

Let's understand what happens step by step.

First:

```
file = open("sample.txt", "r")
```

opens the file in read mode.

Next:

```
content = file.read()
```

reads the complete contents of the file and stores them in the variable `content`.

Then:

```
print(content)
```

displays the content on the screen.

Finally:

```
file.close()
```

closes the file.

The output will be:

Hello

Welcome to Python

The important point is that `read()` reads the **entire available content** from the file.

Reading One Line Using `readline()`

Sometimes we don't want to read the complete file at once. We may want to read only one line.

For this, we can use `readline()`.

```
file = open("sample.txt", "r")
```

```
print(file.readline())
```

```
file.close()
```

If the file contains:

Hello

Welcome to Python

the output will be:

Hello

The first `readline()` reads the first line.

If we call `readline()` again:

```
file = open("sample.txt", "r")
```

```
print(file.readline())
```

```
print(file.readline())
```

```
file.close()
```

the first call reads:

Hello

and the second call reads:

Welcome to Python

This happens because Python keeps track of its current position in the file while reading.

So remember:

`read()` → reads the entire file

`readline()` → reads one line at a time

Reading All Lines Using `readlines()`

The `readlines()` function reads all the lines from a file and returns them as a **list**.

For example:

```
file = open("sample.txt", "r")
```

```
print(file.readlines())
```

```
file.close()
```

If the file contains:

Hello

Welcome to Python

the output will be similar to:

```
['Hello\n', 'Welcome to Python']
```

Here, each line becomes one element in the list.

The `\n` represents the **newline character**. It indicates that the first line ended and the next line started.

So, `readlines()` is useful when we want to work with the file **line by line as a collection of strings**.

Difference Between `read()`, `readline()` and `readlines()`

These three functions all read data, but they work differently.

`read()` reads the **complete file** as one string.

`readline()` reads **one line** at a time.

`readlines()` reads all lines and gives them as a **list of strings**.

For example, if the file contains:

Hello

Welcome to Python

then:

```
file.read()
```

gives the complete content.

```
file.readline()
```

gives one line.

```
file.readlines()
```

gives something similar to:

```
['Hello\n', 'Welcome to Python']
```

Writing Data to a File

We can use the `write()` function to store information in a file.

For example:

```
file = open("sample.txt", "w")
```

```
file.write("I am learning Python")
```

```
file.close()
```

The `open()` statement opens the file in "w" mode. The `write()` function sends the text "I am learning Python" into the file.

After the program runs, the file contains:

I am learning Python

One important point is that `write()` does not automatically add a new line after the text.

For example:

```
file.write("Hello")
```

```
file.write("Python")
```

may result in:

HelloPython

If we want them on separate lines, we can use `\n`:

```
file.write("Hello\n")
```

```
file.write("Python")
```

The file will contain:

Hello

Python

Overwriting Existing File Content

Suppose `sample.txt` already contains:

Old information

Now we run:

```
file = open("sample.txt", "w")  
file.write("New information")  
file.close()
```

The old content is removed.

The file becomes:

New information

This happens because we used "w" mode.

Therefore, be careful when using "w" mode. If you want to **keep the existing information and add something new**, use "a" instead.

Appending Data to a File

Appending means **adding new information to the existing information**.

Suppose the file contains:

I am learning Python

We can add another line using:

```
file = open("sample.txt", "a")
```

```
file.write("\nPython is easy")
```

```
file.close()
```

The file now contains:

I am learning Python

Python is easy

The original line was not removed.

The difference between "w" and "a" is therefore very important:

"w" → Old content is replaced

"a" → Old content is kept and new content is added

Closing a File

After finishing our work with a file, we should close it.

We use:

```
file.close()
```

For example:

```
file = open("sample.txt", "r")
```

```
content = file.read()
```

```
print(content)
```

```
file.close()
```

The `close()` statement tells Python that we have finished working with the file.

Closing a file is important because it releases the resources being used by the opened file and ensures that changes are properly handled.

Think of it like **closing a notebook after finishing your work**.

Using `with open()`

Python provides a better and safer way to work with files using the `with` statement.

Instead of writing:

```
file = open("sample.txt", "r")
```

```
print(file.read())
```

```
file.close()
```

we can write:

```
with open("sample.txt", "r") as file:
```

```
    print(file.read())
```

Here, Python opens the file and stores the opened file in the variable `file`.

The indented code:

```
print(file.read())
```

runs while the file is open.

After the `with` block finishes, Python automatically closes the file.

Therefore, we don't need to write:

```
file.close()
```

when using `with open()`.

This is generally the preferred way to work with files because it automatically handles closing the file.

Understanding `with open()` Step by Step

Consider:

`with open("sample.txt", "r") as file:`

```
    data = file.read()
```

```
    print(data)
```

The first part:

```
open("sample.txt", "r")
```

opens `sample.txt` in read mode.

The part:

```
as file
```

gives us a variable named `file` that represents the opened file.

The indented code:

```
data = file.read()
```

reads the contents and stores them in `data`.

Then:

```
print(data)
```

displays the contents.

Once Python reaches the end of the `with` block, it automatically closes the file.

So we can think of it as:

Open file



Work with file



Finish block



Python automatically closes file

Simple Real-Life Example: Saving a Student Name

Suppose we want to save a student's name in a file.

We can write:

with open("student.txt", "w") as file:

```
    file.write("Aniketh")
```

Here, `"student.txt"` is the file name and `"w"` means that we want to write data.

The statement:

```
file.write("Aniketh")
```

stores the text `"Aniketh"` inside the file.

After the program finishes, `student.txt` contains:

Aniketh

Because we used `with`, Python automatically closes the file.

Reading the Student Name

Now suppose we want to read the student's name from the file.

with open("student.txt", "r") as file:

```
    name = file.read()
```

```
    print(name)
```

Python opens the file in read mode. The `read()` function takes the content `"Aniketh"` from the file and stores it in the variable `name`.

Then `print(name)` displays:

Aniketh

So the complete process is:

Python program



Write "Aniketh"



student.txt



Read "Aniketh"



Display on screen

Lambda Function

A **lambda function** is a small function that is usually written in **one line** and does not have a normal function name created using **def**.

A normal function can be written as:

```
def add(a, b):  
    return a + b
```

We can write the same simple operation using a lambda function:

```
add = lambda a, b: a + b
```

Here, **lambda** tells Python that we are creating a lambda function. **a** and **b** are the inputs, and **a + b** is the operation that produces the result.

We can then call it:

```
print(add(5, 3))
```

The value **5** goes into **a**, and **3** goes into **b**.

The function performs:

$5 + 3 = 8$

Therefore, the output is:

8

Lambda functions are useful when we need a **small, simple function for a short operation**, especially when working with functions such as **map()**, **filter()**, and **reduce()**.

map() Function

The `map()` function is used when we want to **apply the same operation to every element in a sequence**, such as a list.

Suppose we have:

```
nums = [1, 2, 3, 4]
```

We want to find the square of every number.

Instead of writing four separate calculations, we can use `map()`:

```
result = list(map(lambda x: x ** 2, nums))
```

```
print(result)
```

The lambda function is:

```
lambda x: x ** 2
```

This means:

Take one value called `x` and calculate `x` raised to the power of 2.

Python applies this operation to every value in `nums`.

The process is:

$$1 \rightarrow 1^2 \rightarrow 1$$
$$2 \rightarrow 2^2 \rightarrow 4$$
$$3 \rightarrow 3^2 \rightarrow 9$$
$$4 \rightarrow 4^2 \rightarrow 16$$

Therefore:

```
[1, 4, 9, 16]
```

The output is:

```
[1, 4, 9, 16]
```

The `map()` function is therefore useful when we want to **transform every item in a collection in the same way**.

filter() Function

The `filter()` function is used when we want to **select only the elements that satisfy a condition**.

Suppose we have:

```
nums = [1, 2, 3, 4, 5]
```

We want only the even numbers.

We can write:

```
result = list(filter(lambda x: x % 2 == 0, nums))
```

```
print(result)
```

Here:

```
lambda x: x % 2 == 0
```

checks whether a number is even.

The `%` operator gives the remainder after division.

For example:

```
2 % 2 → 0
```

```
4 % 2 → 0
```

Therefore, `2` and `4` satisfy the condition.

Python checks each value:

```
1 → odd → reject
```

```
2 → even → keep
```

```
3 → odd → reject
```

```
4 → even → keep
```

```
5 → odd → reject
```

The result is:

```
[2, 4]
```

So remember:

`map()` → change/transform every item

`filter()` → select items that satisfy a condition

`zip()` Function

The `zip()` function is used to **combine corresponding elements from two or more sequences**.

Suppose we have:

```
names = ["Chinna", "Ram"]
```

```
marks = [80, 90]
```

The first name is "Chinna" and the first mark is 80. The second name is "Ram" and the second mark is 90.

We can combine them using:

```
result = list(zip(names, marks))
```

```
print(result)
```

The result is:

```
[('Chinna', 80), ('Ram', 90)]
```

Python combines the values according to their positions:

Chinna → 80

Ram → 90

So the resulting pairs are:

```
("Chinna", 80)
```

```
("Ram", 90)
```

This is useful when we have related information stored in separate lists and want to combine the corresponding values.

For example:

Names list + Marks list

↓	↓
Chinna	80
Ram	90

↓

Combined pairs

reduce() Function

The `reduce()` function is used when we want to repeatedly apply an operation to the values and finally produce **one result**.

Unlike `map()`, which produces a result for each element, `reduce()` keeps combining values until only one final value remains.

The `reduce()` function is available in the **functools module**, so we first need to import it.

```
from functools import reduce
```

Suppose we have:

```
nums = [1, 2, 3, 4]
```

and we want to calculate their total.

We can write:

```
from functools import reduce
```

```
nums = [1, 2, 3, 4]
```

```
result = reduce(lambda x, y: x + y, nums)
```

```
print(result)
```

The addition happens step by step.

First:

$1 + 2 = 3$

Then:

$3 + 3 = 6$

Then:

$6 + 4 = 10$

The final result is:

10

So `reduce()` takes several values and repeatedly combines them until a **single final value** is produced.

Think of it like:

[1, 2, 3, 4]

↓

1 + 2

↓

3 + 3

↓

6 + 4

↓

10

The important point is that `reduce()` works **cumulatively, step by step**.

Difference Between `map()`, `filter()`, `zip()` and `reduce()`

These functions have different purposes.

`map()` applies an operation to every element.

[1, 2, 3]

↓

square each value

↓

[1, 4, 9]

`filter()` checks a condition and keeps only the values that satisfy it.

[1, 2, 3, 4]

↓

keep even numbers

↓

[2, 4]

`zip()` combines corresponding values from different sequences.

Names: Chinna Ram

Marks: 80 90

↓

(Chinna,80)

(Ram,90)

`reduce()` repeatedly combines values and produces one final result.

[1, 2, 3, 4]

↓

add step-by-step



10

A simple way to remember them is:

map() → Apply to all

filter() → Select some

zip() → Combine

reduce() → Produce one final value

Complete Quick Revision

File I/O is used to read data from files and write data into files. We use `open()` to open a file, modes such as `"r"`, `"w"`, and `"a"` to specify what we want to do, `read()` to read the complete content, `readline()` to read one line, `readlines()` to read all lines as a list, and `write()` to store data in a file. `close()` closes the file, while `with open()` automatically handles closing.

Lambda is a small one-line function used for simple operations. `map()` applies an operation to every element, `filter()` selects elements according to a condition, `zip()` combines corresponding elements from different sequences, and `reduce()` repeatedly combines elements to produce one final result.

File I/O



Read / Write files

lambda



Small one-line function

map()



Apply operation to every item

filter()



Select items based on condition

zip()



Combine corresponding items

`reduce()`



Combine step-by-step → One final result

This version is designed so that students can understand **what the code means, what happens to the data, and why each function behaves the way it does**, rather than only memorizing definitions.